

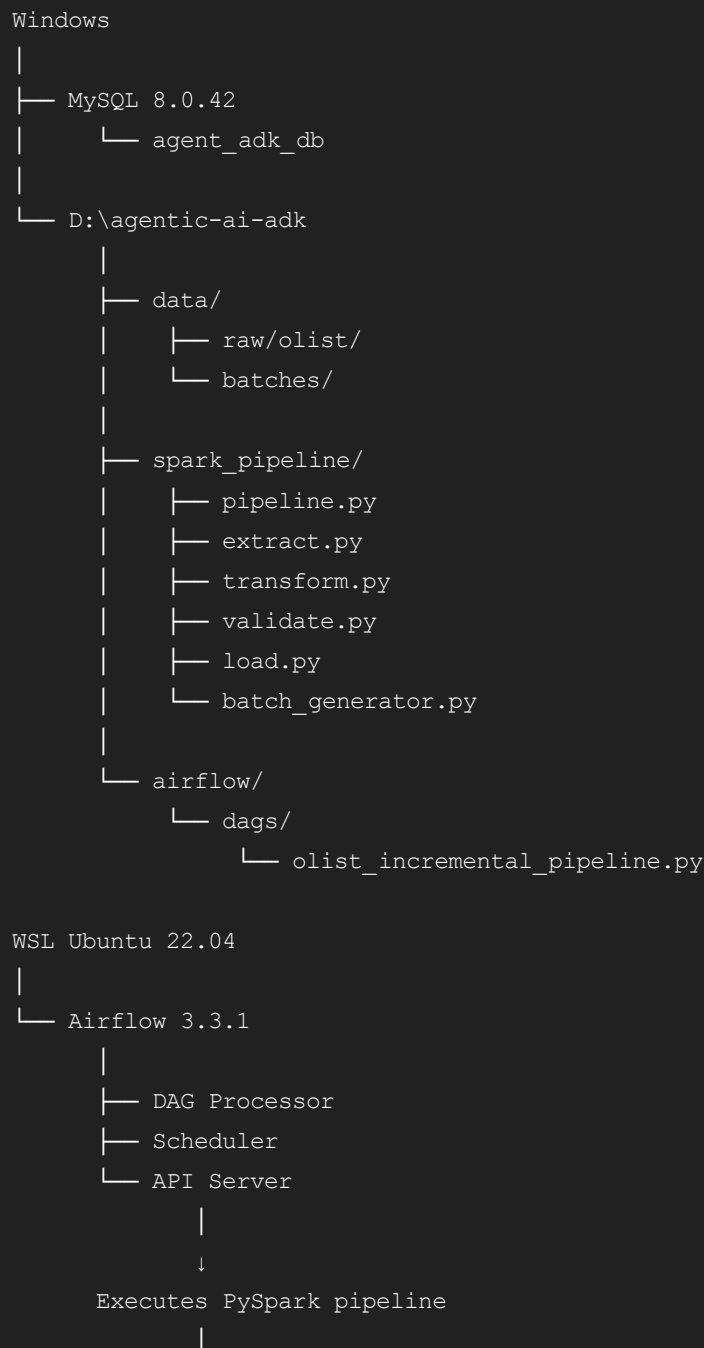
Yes. This is worth documenting as a **runbook**, because tomorrow you should be able to start from a clean WSL session and operate the pipeline without remembering today's troubleshooting.

Below is the complete **Airflow + WSL + PySpark + MySQL operational guide**, including what each command does, why we needed it, the problems we encountered, and the exact procedure for triggering the DAG again.

Airflow + WSL Incremental Pipeline — Complete Runbook

1. Overall architecture

Your current pipeline is:



↓
Windows MySQL

The important concept is:

Airflow orchestrates. PySpark processes. MySQL stores.

Airflow is not itself reading every CSV and inserting rows.

2. Why WSL was required

Airflow does not officially support native Windows execution.

Therefore:

```
Windows
  ↓
WSL 2
  ↓
Ubuntu 22.04
  ↓
Airflow
```

Your actual project files remain on Windows:

```
D:\agentic-ai-adk
```

WSL accesses them as:

```
/mnt/d/agentic-ai-adk
```

So both environments see the same project.

3. WSL setup

Your WSL installation:

```
Ubuntu 22.04.2 LTS
Python 3.10.6
WSL 2
```

Check WSL from Windows CMD/PowerShell:

bat

```
wsl -l -v
```

Expected:

```
Ubuntu-22.04    Stopped/Running    2
```

Start Ubuntu:

bat

```
wsl
```

Or:

bat

```
wsl -d Ubuntu-22.04
```

4. Java requirement

PySpark requires Java.

We installed Java 17 inside WSL:

Bash

```
sudo apt update  
sudo apt install -y openjdk-17-jdk
```

Check:

Bash

```
java -version
```

Expected:

```
openjdk version "17..."
```

Find Java:

Bash

```
readlink -f $(which java)
```

Your Java location is:

```
/usr/lib/jvm/java-17-openjdk-amd64
```

Persistent configuration was added to:

```
~/.bashrc
```

Bash

```
export JAVA_HOME=/usr/lib/jvm/java-17-openjdk-amd64
export PATH=$JAVA_HOME/bin:$PATH
export PYSPARK_PYTHON=/home/suresh/airflow/venv/bin/python
export PYSPARK_DRIVER_PYTHON=/home/suresh/airflow/venv/bin/python
```

After opening a new WSL terminal, these are automatically loaded.

You can verify:

Bash

```
echo $JAVA_HOME
java -version
```

5. Airflow virtual environment

Airflow has its own WSL Python environment:

```
/home/suresh/airflow/venv
```

Activate it:

Bash

```
source ~/airflow/venv/bin/activate
```

Your prompt should become something like:

```
(venv) suresh@mrdsuresh:...
```

Verify:

Bash

```
which python
```

Expected:

```
/home/suresh/airflow/venv/bin/python
```

6. Airflow installation

We installed:

Bash

```
pip install --upgrade pip
pip install "apache-airflow==3.3.1"
pip install pyspark==4.0.1 mysql-connector-python python-dotenv
```

Versions:

```
Airflow 3.3.1
PySpark 4.0.1
MySQL Connector
python-dotenv
```

Check:

Bash

```
airflow version
```

7. Why Airflow has its own database

This is important.

Airflow uses:

```
/home/suresh/airflow/airflow.db
```

with:

SQLite

This database stores Airflow metadata such as:

- DAGs
- DAG runs
- task states
- scheduler information
- task execution metadata

It is **not your business database**.

Your business data is in:

agent_adk_db

on Windows MySQL.

So:

Airflow metadata

↓

airflow.db

Olist/business data

↓

agent_adk_db

They are separate.

8. Project location inside WSL

Your Windows project:

D:\agentic-ai-adk

becomes:

/mnt/d/agentic-ai-adk

Go there:

Bash

```
cd /mnt/d/agent-ai-adk
```

Check:

Bash

```
ls
```

You should see things like:

```
api_server
airflow
data
mcp_server
spark_pipeline
.env
requirements.txt
```

9. Airflow DAG location

Airflow expects DAGs in:

```
/home/suresh/airflow/dags
```

Your actual DAG is maintained in the project:

```
/mnt/d/agent-ai-adk/airflow/dags/olist_incremental_pipeline.py
```

We created a symbolic link:

Bash

```
mkdir -p ~/airflow/dags
ln -sf /mnt/d/agent-ai-adk/airflow/dags/olist_incremental_pipeline.py \
~/airflow/dags/olist_incremental_pipeline.py
```

Therefore:

```
Airflow DAG folder
    ↓
/home/suresh/airflow/dags/
    ↓ symlink
Project DAG
```

↓
`/mnt/d/agentic-ai-adk/airflow/dags/`

This means you edit the project file, not a duplicate.

Check:

Bash

```
ls -l ~/airflow/dags/
```

You should see:

```
olist_incremental_pipeline.py -> /mnt/d/agentic-ai-adk/airflow/dags/olist_incremen
```

10. WSL → Windows MySQL connection

This was one of the important problems we solved.

Your MySQL is running on Windows:

```
localhost:3306
```

But from WSL:

```
localhost
```

means the **WSL machine itself**, not Windows.

Therefore this failed initially:

```
WSL → localhost:3306
```

We discovered the Windows host IP dynamically using:

Bash

```
ip route show default
```

For example:

```
default via 172.xx.xx.1 dev eth0
```

The address after `via` is the Windows host gateway.

11. MySQL user permissions

Initially WSL could reach MySQL but MySQL rejected the user:

```
Access denied for user 'agent_adk_user'@'172....'
```

This proved:

```
WSL → Windows MySQL
```

network connectivity was working.

The problem was MySQL user authorization.

We fixed it in MySQL Workbench with:

SQL

```
CREATE USER IF NOT EXISTS 'agent_adk_user'@'%'
IDENTIFIED BY 'YOUR_EXISTING_PASSWORD';

GRANT ALL PRIVILEGES ON agent_adk_db.*
TO 'agent_adk_user'@'%';

FLUSH PRIVILEGES;
```

The `%` means the MySQL user can connect from the WSL host rather than only from `localhost`.

12. Important `.env` configuration

Your project `.env` remains:

env

```
DB_HOST=localhost
DB_PORT=3306
DB_USER=agent_adk_user
DB_PASSWORD=...
DB_NAME=agent_adk_db
```

Do not change `DB_HOST=localhost` **permanently.**

Why?

Because when the PySpark pipeline runs directly on Windows:

```
Windows Python
    ↓
localhost:3306
    ↓
Windows MySQL
```

works correctly.

But Airflow is running inside WSL:

```
WSL Airflow
    ↓
Windows MySQL
```

so the DAG dynamically determines the Windows host IP and overrides `DB_HOST` only for that child process.

This was an important design decision.

13. Airflow API Server problem we encountered

Airflow 3.3 uses an API server as part of task execution.

Our first DAG execution failed with:

```
httpx.ConnectError: [Errno 111] Connection refused
```

The task worker was trying to communicate with the Airflow API server.

We discovered that the API server needed to be running.

The API server is started with:

```
Bash

airflow api-server
```

It listens on:

```
0.0.0.0:8080
```

Check:

Bash

```
ss -ltnp | grep 8080
```

Expected:

```
LISTEN ... 0.0.0.0:8080 ...
```

14. Airflow components that must run

For your current setup, you need these processes:

Terminal 1 — DAG Processor

Bash

```
airflow dag-processor
```

Terminal 2 — Scheduler

Bash

```
airflow scheduler
```

Terminal 3 — API Server

Bash

```
airflow api-server
```

These should remain running.

The roles are:

```
DAG Processor
  ↓
discovers/parses DAG Python files

Scheduler
  ↓
decides when tasks should execute

API Server
```

15. API connection problem we solved

Even after starting the API server, task workers were attempting:

```
localhost:8080
```

and we encountered:

```
Connection refused
```

We configured:

Bash

```
export AIRFLOW__CORE__EXECUTION_API_SERVER_URL=http://127.0.0.1:8080/execution/
```

and persisted it:

Bash

```
echo 'export AIRFLOW__CORE__EXECUTION_API_SERVER_URL=http://127.0.0.1:8080/execution/' >
```

Then:

Bash

```
source ~/.bashrc
```

This fixed the worker → API server communication.

Check:

Bash

```
echo $AIRFLOW__CORE__EXECUTION_API_SERVER_URL
```

Expected:

```
http://127.0.0.1:8080/execution/
```

16. Checking Airflow health

With the API server running:

Bash

```
curl http://127.0.0.1:8080/api/v2/monitor/health
```

You want:

JSON

```
{
  "metadatabase": {
    "status": "healthy"
  },
  "scheduler": {
    "status": "healthy"
  },
  "dag_processor": {
    "status": "healthy"
  }
}
```

This is a very useful diagnostic command.

17. Graphviz warning

You repeatedly saw:

```
Could not import graphviz.
Rendering graph to the graphical format will not be possible.
```

This is **not a pipeline failure**.

It only means Airflow cannot render graphical DAG representations.

You can safely ignore it for now.

18. DAG import error problem

At one point we had:

```
TabError: inconsistent use of tabs and spaces in indentation
```

This caused:

```
has_import_errors | True
```

and the DAG became stale.

The fix was to replace the entire DAG file with clean, consistent indentation.

Before running a DAG, you can test Python syntax:

Bash

```
python -m py_compile olist_incremental_pipeline.py
```

If there is no output:

```
Python syntax = OK
```

Then:

Bash

```
airflow dags list-import-errors
```

Expected:

```
No data found
```

Then:

Bash

```
airflow dags details olist_incremental_pipeline
```

Check:

```
is_paused      | False
is_stale       | False
has_import_errors | False
```

This is an excellent **pre-flight check**.

19. Airflow paused DAG problem

Another problem we encountered was:

```
is_paused | True
```

A paused DAG will not execute normally.

Fix:

Bash

```
airflow dags unpause olist_incremental_pipeline
```

Verify:

Bash

```
airflow dags details olist_incremental_pipeline
```

You want:

```
is_paused | False
```

20. The XCom problem we encountered

The pipeline itself was actually working.

We had:

```
find_next_batch → SUCCESS  
run_pipeline    → SUCCESS  
verify_batch    → FAILED
```

The error was:

```
XCom not found
```

and:

```
No pipeline record found for None
```

The reason was that:

Python

Run

```
run_pipeline(batch_id)
```

didn't return the batch ID.

We fixed it with:

Python

Run

```
return batch_id
```

Therefore Airflow can pass:

```
find_next_batch
  ↓
batch_id
  ↓
run_pipeline
  ↓
return batch_id
  ↓
verify_batch
```

This is why your current version works.

21. Current DAG logic

Your DAG essentially does this:

```
find_next_batch()
|
├─ Scan data/batches/
|
├─ Find batches
|
├─ Check pipeline_batches
|
├─ Select first unprocessed batch
|
↓
run_pipeline(batch_id)
|
├─ Run PySpark
├─ Load MySQL
├─ Refresh analytics
├─ Mark batch SUCCESS
├─ Update watermark
|
↓
```

```
verify_batch(batch_id)
|
└─ Confirm MySQL status = SUCCESS
```

22. What actually happens when you trigger the DAG

When you execute:

Bash

```
airflow dags trigger olist_incremental_pipeline
```

Airflow creates a new DAG run.

Then:

Task 1

```
find_next_batch
```

looks at:

```
data/batches/
```

For example:

```
batch_201609
batch_201610
batch_201612
...
```

and checks:

SQL

```
SELECT batch_id
FROM pipeline_batches
WHERE status = 'SUCCESS';
```

It selects the earliest batch that hasn't successfully completed.

Task 2

Airflow executes approximately:

Bash

```
python -m spark_pipeline.pipeline \  
    --batch-id batch_YYYYMM
```

The PySpark pipeline:

```
reads batch CSV  
    ↓  
Spark DataFrame  
    ↓  
transform  
    ↓  
validate  
    ↓  
MySQL
```

Task 3

`verify_batch` checks:

SQL

```
SELECT status  
FROM pipeline_batches  
WHERE batch_id = %s;
```

and expects:

SUCCESS

If yes:

`verify_batch` → SUCCESS

23. What your successful run proved

Your latest run:

manual__2026-09-09T15:56:02.490031+00:00

produced:

```
find_next_batch | success
run_pipeline    | success
verify_batch    | success
```

This proves:

```
Airflow
  ↓
Scheduler
  ↓
Task worker
  ↓
PySpark
  ↓
Windows MySQL
  ↓
pipeline metadata
  ↓
verification
```

is functioning end-to-end.

24. Tomorrow — exact startup procedure

This is the section you will probably use most.

Step 1 — Open WSL

From Windows:

```
bat
wsl
```

Step 2 — Go to project

```
Bash
cd /mnt/d/agent-ai-adk
```

Step 3 — Activate Airflow environment

Bash

```
source ~/airflow/venv/bin/activate
```

Check:

Bash

```
which python
```

Expected:

```
/home/suresh/airflow/venv/bin/python
```

Step 4 — Verify Java

Bash

```
java -version
```

Expected Java 17.

Also:

Bash

```
echo $JAVA_HOME
```

Step 5 — Verify Airflow API environment

Bash

```
echo $AIRFLOW__CORE__EXECUTION_API_SERVER_URL
```

Expected:

```
http://127.0.0.1:8080/execution/
```

If blank, run:

Bash

```
source ~/.bashrc
```

25. Start the three Airflow services

You need three WSL terminals.

Terminal 1

Bash

```
source ~/airflow/venv/bin/activate  
airflow dag-processor
```

Leave it running.

Terminal 2

Bash

```
source ~/airflow/venv/bin/activate  
airflow scheduler
```

Leave it running.

Terminal 3

Bash

```
source ~/airflow/venv/bin/activate  
airflow api-server
```

Leave it running.

26. Check Airflow health

From another terminal:

Bash

```
source ~/airflow/venv/bin/activate
```

Then:

Bash

```
curl http://127.0.0.1:8080/api/v2/monitor/health
```

Make sure:

```
metadatabase = healthy
scheduler    = healthy
dag_processor = healthy
```

27. Check the DAG before triggering

Bash

```
cd /mnt/d/agent-ai-adk/airflow/dags
```

Syntax:

Bash

```
python -m py_compile olist_incremental_pipeline.py
```

No output = good.

Then:

Bash

```
airflow dags list-import-errors
```

Expected:

```
No data found
```

Then:

Bash

```
airflow dags details olist_incremental_pipeline
```

Check:

```
is_paused      | False
is_stale       | False
```

```
has_import_errors | False
```

If all three are correct, you're ready.

28. Trigger the DAG tomorrow

Simply run:

Bash

```
airflow dags trigger olist_incremental_pipeline
```

You will see something like:

```
run_id = manual__2026-09-10T...  
state = queued
```

Queued is normal initially.

The scheduler should then pick it up.

29. Check task states

Take the `run_id` returned by the trigger command.

For example:

```
manual__2026-09-10T10:00:00+00:00
```

Then:

Bash

```
airflow tasks states-for-dag-run \  
olist_incremental_pipeline \  
manual__2026-09-10T10:00:00+00:00
```

Expected:

```
find_next_batch | success  
run_pipeline    | success  
verify_batch    | success
```

That is your primary confirmation.

30. What happens on the next trigger?

This is important.

Suppose:

```
batch_201609 → SUCCESS
batch_201610 → SUCCESS
batch_201612 → SUCCESS
```

When you trigger again, the DAG doesn't intentionally start from `batch_201609`.

It checks:

SQL

```
pipeline_batches
```

and finds the batches already marked:

```
SUCCESS
```

Then selects the next pending batch.

Therefore:

```
Trigger #1
  ↓
batch_201609

Trigger #2
  ↓
batch_201610

Trigger #3
  ↓
batch_201612

Trigger #4
  ↓
next pending batch
```

This is how we're simulating incremental arrivals from the historical Olist dataset.

31. What if there are no pending batches?

Eventually:

```
All 25 batches
```

↓

```
SUCCESS
```

Then:

```
find_next_batch
```

raises:

```
No pending batches available.
```

That is **not a pipeline bug**. It simply means the simulated source has been completely consumed.

32. Do NOT regenerate the batches every day

You already ran:

```
Bash
```

```
python -m spark_pipeline.batch_generator
```

and generated the 25 monthly batches.

You do not need to run this every day.

It was a one-time preparation step:

```
Raw Olist CSV
```

↓

```
batch_generator
```

↓

```
25 chronological batches
```

After that:

```
Airflow
```

↓

```
one batch at a time
```

33. Do NOT manually import CSVs into MySQL

The intended architecture is:

```
CSV batch
  ↓
PySpark
  ↓
MySQL
```

not:

```
CSV
  ↓
Workbench import
```

This keeps the ingestion process reproducible and demonstrates actual data-engineering orchestration.

34. Important distinction: Airflow vs PySpark

Think of them this way:

Airflow

Answers:

When should this happen, and in what order?

PySpark

Answers:

How do I process and transform this data?

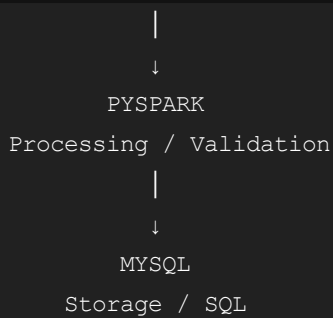
MySQL

Answers:

Where does the processed data live?

Therefore:

```
AIRFLOW
Orchestration
```



35. Current project status

At this point, you have completed:

- [✓] Olist dataset
- [✓] Raw data extraction
- [✓] Monthly batch generation
- [✓] PySpark processing
- [✓] Data validation
- [✓] MySQL schema
- [✓] Incremental loading
- [✓] Pipeline metadata
- [✓] Watermark mechanism
- [✓] Analytics tables
- [✓] WSL 2
- [✓] Java 17
- [✓] Airflow 3.3.1
- [✓] Airflow DAG Processor
- [✓] Airflow Scheduler
- [✓] Airflow API Server
- [✓] WSL → Windows MySQL connectivity
- [✓] DAG discovery
- [✓] DAG import validation
- [✓] Batch selection
- [✓] PySpark execution through Airflow
- [✓] MySQL ingestion through Airflow
- [✓] Batch verification
- [✓] End-to-end successful DAG run

The key achievement is:

You now have a functioning incremental data pipeline where Airflow selects the next historical Olist batch, invokes PySpark, PySpark loads the batch into Windows MySQL, analytics are refreshed, and Airflow verifies the result.

36. Your shortest "tomorrow" cheat sheet

If everything was shut down overnight, use this:

Bash

```
# 1. Start WSL
wsl

# 2. Activate Airflow
source ~/airflow/venv/bin/activate

# 3. Go to project
cd /mnt/d/agent-ai-adk

# 4. Start DAG Processor
airflow dag-processor
```

Open another WSL terminal:

Bash

```
source ~/airflow/venv/bin/activate
cd /mnt/d/agent-ai-adk
airflow scheduler
```

Open another WSL terminal:

Bash

```
source ~/airflow/venv/bin/activate
cd /mnt/d/agent-ai-adk
airflow api-server
```

Open another WSL terminal:

Bash

```
source ~/airflow/venv/bin/activate
cd /mnt/d/agent-ai-adk/airflow/dags

python -m py_compile olist_incremental_pipeline.py

airflow dags list-import-errors

airflow dags details olist_incremental_pipeline
```

If:

```
is_paused      | False
is_stale       | False
```

```
has_import_errors | False
```

then:

Bash

```
airflow dags trigger olist_incremental_pipeline
```

Copy the returned `run_id`, then:

Bash

```
airflow tasks states-for-dag-run \  
olist_incremental_pipeline \  
<YOUR_RUN_ID>
```

Success:

```
find_next_batch | success  
run_pipeline    | success  
verify_batch    | success
```

That's the complete daily operating procedure.

One correction to keep in mind from today's troubleshooting: **do not use** `airflow dags list-runs --dag-id ...` with your Airflow 3.3.1 CLI; your installation rejected that syntax. The `states-for-dag-run` command above is the reliable command we've already successfully used in your environment.